Homework — 1.4.2 Data Structures — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Total: Part A 14 + Part B 6 = 20 marks.

Part A

1. [4] Stack trace (top listed first): - push(8) → [8] - push(3) → [3, 8] - push(6) → [6, 3, 8] - pop() → returns **6** → [3, 8] - pop() → returns **3** → [8] - push(1) → [1, 8]

Values popped (in order): **6, then 3**. Final stack (top → bottom): **1, 8**. *Mark: pushes added to the top in correct order (1); first pop returns 6 (1); second pop returns 3 (1); correct final contents 1, 8 with 1 on top (1).*

2. [3] In a simple linear queue, once items have been dequeued from the **front** the spaces at the start of the array **cannot be reused** — the rear pointer keeps moving toward the end, so the queue can report "full" even though earlier slots are empty (wasted space) (1). A circular queue treats the array as a ring: the rear pointer is updated with `rear = (rear + 1) MOD size`, so when it reaches the end it **wraps back to the start** and reuses the freed slots (1). This makes more efficient use of the fixed memory without needing to shuffle/shift all the elements down (1). *(AO1)*

3. [3] BST after inserting 40, 60, 25, 35, 10, 50:

```
        40
       /  \
     25    60
    /  \   /
  10   35 50
```

- 40 is the root.
- 60 > 40 → right of 40; 25 < 40 → left of 40.
- 35 > 25 (and < 40) → right of 25; 10 < 25 → left of 25.
- 50 < 60 (and > 40) → left of 60. *Mark: 40 root with 25 left / 60 right (1); 10 left and 35 right of 25 (1); 50 as left child of 60 (1).*

4. [2] In-order traversal = **Left, Root, Right**: **10, 25, 35, 40, 50, 60** (1). Special property: an in-order traversal of a **BST** visits the nodes in **ascending (sorted) order** (1). *(AO2)*

5. [2] 23 MOD 10 = **3**; 57 MOD 10 = **7**; 33 MOD 10 = **3** (1). **Collision:** 33 hashes to index 3, which is already occupied by 23 (1 — for identifying the collision and naming a resolution). Resolution method (any one): **linear probing** (place at the next free slot) **or chaining** (store a linked list of keys at that index). *Mark: all three indices correct 3, 7, 3 (1); collision at index 3 identified AND a valid resolution method named (1).*

Part B

6. [2] Interrupts can occur while another interrupt is already being serviced (nested), so the CPU must be able to return to each task in the correct order (1). A stack is **LIFO**, so the registers (e.g. PC and ACC) of the interrupted task are **pushed** on, and when the higher-priority ISR finishes they are **popped** off in reverse order — the most recently interrupted task resumes first — guaranteeing every task resumes correctly even with several interrupts nested (1). *(AO1)*

7. [1] The control bus carries **control signals** (e.g. read/write signals, clock pulses, interrupt requests) between the CPU and other components to coordinate/synchronise their operation. *(AO1)*

8. [3] A foreign key is an attribute in one table that is the **primary key of another table**, used to **link** the two tables together (1). Referential integrity ensures that every foreign-key value **matches a valid existing primary key** in the related table (or is null) (1), so there are **no "orphaned" records** that reference a row which does not exist (1). *(AO1)*